

After loading the image of the ball, we call the `get_rect()` method ❶ to get the bounding rectangle of the image. That call returns a `rect` object, which we store into a variable called `ballRect`. We use `ballRect.width` and `ballRect.height` to get direct access to the width and height of the ball image. (In the previous version, we used a constant of 100 for the width and the height.) Getting these values from the image that was loaded makes our code much more adaptable because it means we can use a graphic of any size.

The code also uses the attributes of the rectangle rather than using separate variables for checking if any part of the ball's rectangle goes over an edge. We can use `ballRect.left` and `ballRect.right` to see if the `ballRect` is off the left or right edges ❷. We do a similar test with `ballRect.top` and `ballRect.bottom`. Rather than updating individual x- and y-coordinate variables, we update the `left` and `top` of the `ballRect`.

The other subtle but important change is in the call to draw the ball ❸. The second argument in the call to `blit()` can be either an (x, y) tuple or a `rect`. The code inside `blit()` uses the left and top position in the `rect` as the x- and y-coordinates.

Playing Sounds

There are two types of sounds that you might want to play in your programs: short sound effects and background music.

Playing Sound Effects

All sound effects must live in external files and must be in either `.wav` or `.ogg` format. Playing a relatively short sound effect consists of two steps: load the sound from an external sound file once; then at the appropriate time(s) play your sound.

To load a sound effect into memory, you use a line like this:

```
<soundVariable> = pygame.mixer.Sound(<path to sound file>)
```

To play the sound effect, you only need to call its `play()` method: